Apex3D: Hackathon Technical Solution Report

Nov 7, 2026 · @Cloude

Apex3D identifies the make, model and year of a car in a photo, at 92.2% Top-1 accuracy on 8,041 unseen Stanford Cars test images. It then helps turn that identity into a draft 3D model.

Field	Details
Problem ID	P5
Problem Title	Intelligent Fine-Grained Vehicle Recognition and Analysis System
Team Name	NoharaFam (Team ID T25)
Team Members	Sinchana R Kambar (1RUA24CSE0461); Sohini Dutta (1RUA24CSE0467); Shreyas Muniraju Kolar (1RUA24CSE0445); Shubanck Prasoon Guptha (1RUA24CSE0453)
GitHub Repository	github.com/sinchanaa23/Apex3D, commit 6260022fded80eb73f265d2bd21d1679a2a09de1 (main, 7 Nov 2026). Update this hash to the final commit before submitting.
Live Demo / Deployment	Demo video: docs/demo/demo.mp4 in the repository. Live app: a free Render web service (render.yaml, lean server without PyTorch); its onrender.com address is to be added once deployed. The full app, with 3D mesh building, runs on the team's GPU laptop.

1. Problem Understanding and Solution Strategy

Core technical challenge. P5 asks for make, model and year across 196 Stanford Cars classes. Classes differ only in small details such as grilles, light shapes and trim, while pose, lighting and background vary far more. The measured result shows it: 63.5% of our model's errors stay within the same make, and 6.2% get the make and model right but the year wrong.

The failure modes we targeted:

- near-identical classes, like neighbouring model years and badge-engineered twins;
- the wrong car being chosen when a photo holds several;
- low light, blur, noise, low resolution and occlusion;
- confident answers for vehicles outside the 196 classes.

Constraints: one RTX 3050 Laptop GPU with 4 GB of VRAM, CPU-only recognition so the GPU stays free for 3D, and results within a few seconds through a public link.

Proposed methodology.

1. A Faster R-CNN detector finds the vehicle, and the app crops the largest confident box.
2. A fine-tuned ConvNeXt-Tiny classifies the crop.
3. Calibrated confidence thresholds sort each answer into confident, ambiguous or rejected, instead of always naming a class.
4. A classical pipeline (CLAHE, Canny, contours, HSV, LBP, HOG, geometry) runs alongside for explanation and comparison.

We kept the deep model as the classifier because, on the same split, it beat the classical baseline by 74.9 points of Top-1. Fusing the two gave no gain on validation.

2. Technical Architecture and Implementation

A photo goes through detection, cropping, classification and a confidence decision. The classical pipeline runs beside it for explanation and comparison, and the confirmed identity drives reference retrieval and optional 3D reconstruction. The system diagram is in docs/architecture_system.png and the request sequence in docs/architecture_sequence.png.

2.1 Classical Algorithmic Baseline

Algorithms utilised (backend/classical_cv/pipeline.py):

- resize to 128 px;
- CLAHE on the LAB lightness channel (clip limit 2.0, 8 × 8 tiles);
- Canny edges and the largest contour, which gives a contour region of interest;
- features: an HSV colour histogram (32 dimensions), LBP texture histograms on a 4 × 4 grid (160), HOG (1,764), and 12 geometric features (aspect ratio, extent, solidity, area share, edge density and 7 log-scaled Hu moments);
- classifier: StandardScaler, PCA to 256 dimensions, and a linear SVM, with C = 0.01 chosen on validation (notebooks/02_classical_baseline.ipynb).

Adaptive selection logic.

- Canny thresholds are computed per image from the median grey level (0.67 × and 1.33 × the median), not fixed.
- CLAHE can be applied adaptively, only when mean luminance falls below 70 out of 255. On validation this changed classical Top-1 by −0.33 points, within noise, so the final classical model does not use it.

2.2 Deep Learning Pipeline

Model architecture.

- Detector: torchvision Faster R-CNN with a MobileNetV3-Large FPN backbone (COCO weights, not retrained). It keeps car, bus, truck and motorcycle boxes scoring at least 0.5. The deployed server runs it as an ONNX export: on 200 test photos its chosen box matches PyTorch's at mean IoU 0.997, and 199 of 200 photos get the same final class.
- Classifier: ConvNeXt-Tiny (28.0 M parameters, ImageNet-22k → 1k weights from timm) at 224 px.
- Why ConvNeXt-Tiny: on the same split it beat EfficientNetV2-S (87.13%) and ResNet-50 (81.58%) on validation and test, and it trains within 2.1 GB of VRAM on the 4 GB laptop GPU (notebooks/01c_baseline_models.ipynb).

Loss functions and training (notebooks/01_classifier_training.ipynb):

- Loss: cross-entropy with label smoothing 0.1.
- Data: Stanford Cars (16,185 images), with a stratified 85/15 split of the training set into 6,922 train and 1,222 validation images. The 8,041 test images were used once, for final reporting.
- Input: the annotated box plus an 8% margin, matching the detector crop used in the app.
- Training: 2 epochs training only the new head, then full fine-tuning with AdamW, layer-wise learning-rate decay 0.9, a cosine schedule and FP16 mixed precision. Early stopping on validation Top-1 picked epoch 28 of 32.
- Augmentation: RandomResizedCrop, horizontal flip, RandAugment, and random erasing, which also simulates occlusion.
- Calibration: temperature scaling (T = 0.617) fitted on validation.
- Optimisation: the deployed model is quantised to static INT8 on its MatMul layers through ONNX Runtime (notebooks/01b_export_quantize.ipynb).

2.3 Hybrid Integration and Adaptive Mechanism

Fusion strategy. We tested feature fusion: the deployed model's 768-dimensional embedding concatenated with each classical feature family, then a linear head (backend/evaluation/classifier_experiments.py). The best fusion on validation, deep plus

geometry, scored 92.09% on test against 92.18% for deep features alone. So the app classifies with the deep model only, and the classical stages are shown as an explanation. We report the fusion as a negative result rather than claim a gain.

Decision logic. The system acts on how sure the calibrated model is:

State	Rule	Share of test images	Accuracy
Confident	top probability ⩾ 0.52 and Top-1 minus Top-2 margin ⩾ 0.34	93.5%	95.0%
Ambiguous	top probability ⩾ 0.52 but margin < 0.34; the Top-5 is shown and the user confirms	1.3%	59.8%
Rejected	top probability < 0.52	5.2%	49.2%

The same rule rejects 91.5% of non-car vehicles and 98.3% of other objects from Caltech-101. Detection is adaptive too: the app crops the largest confident box rather than the top-scoring one, which raised full-pipeline Top-1 from 80.6% to 92.4% on 500 test images.

3. Experimental Benchmarks and Evaluation

The deep model reaches 92.2% Top-1 on the 8,041 unseen test images. The classical baseline reaches 17.3%, and fusing the two adds nothing. PSNR, SSIM and LPIPS in the template measure image enhancement, which P5 does not ask for, so we report P5′s recognition metrics instead. Every number comes from the result files named in eval_P5_T25.md.

3.1 Quantitative Benchmarks

Approach	Top-1 ↑	Top-5 ↑	Macro F1 ↑	Size	Latency (ms / image, CPU)
Classical baseline (HSV, LBP, HOG, geometry + linear SVM)	17.32%	36.81%	15.85%	2.5 MB	15 (i5-1135G7)
Deep learning only (ConvNeXt-Tiny, PyTorch FP32)	92.20%	98.42%	92.08%	112.0 MB	49.2 (i5-12500H)
Deep, deployed (ONNX INT8, MatMul layers)	92.18%	98.51%	not measured	34.6 MB	19.4 (i5-12500H)
Proposed hybrid (deep + geometry, chosen on validation)	92.09%	97.60%	91.95%	35.8 MB	22.6 (mixed machines)

Test set: 8,041 images, annotated-box crops with an 8% margin; chance is 0.51% Top-1. The latencies come from two different laptops, so compare them only within one CPU.

Other P5 metrics:

- **Localisation:** the MobileNetV3 detector has mean IoU 0.916 and mAP@[.5:.95] 0.644 (500 test images). The ResNet-50 detector has 0.931 and 0.686.
- **Full pipeline:** classifying detector crops gives 92.4% Top-1 (500 images).
- **Calibration:** test ECE is 0.024.

Analysis.

- Hand-crafted features cannot separate classes that differ in small trim details: HOG alone carries most of the classical accuracy (14.4% on validation).
- The deep model is 75 points better. Quantisation cuts its size by 69% and its CPU time by 43% against ONNX FP32, for a 0.02-point loss.
- The hybrid adds 3.2 ms and 1.2 MB for no accuracy gain, so the deployed system uses the deep model alone.

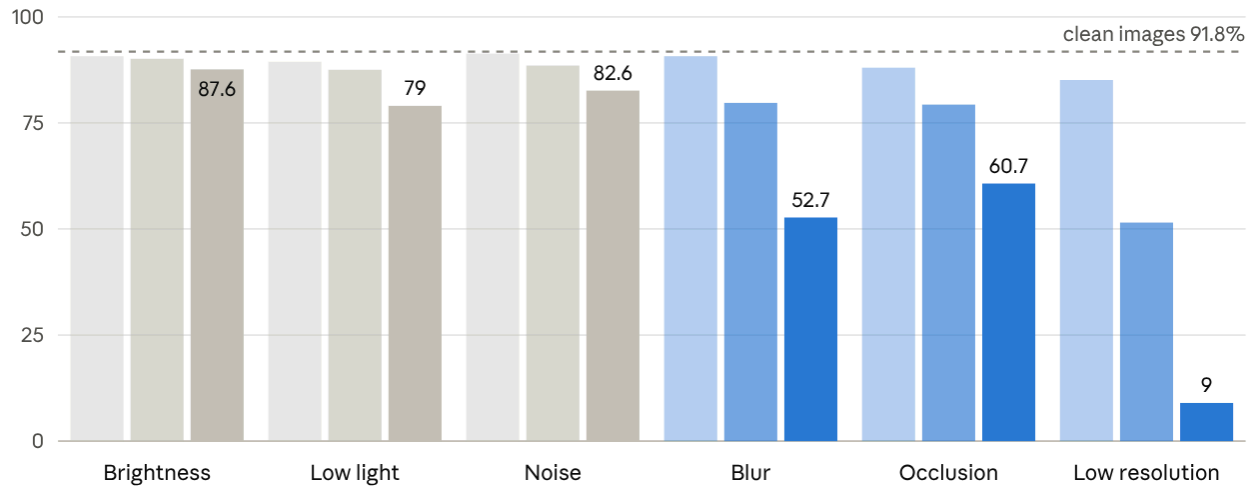
3.2 Visual Outputs and Intermediate Representations

All figures are in the repository:

1. **Intermediate classical stages** for one photo (original, CLAHE, Canny edges, contours, contour ROI, HOG): results/figures/classical_stages.png. The app's Classical page shows these live.
2. **Detection and identity output** with the bounding box, Top-5 and confidence state: docs/screenshots/02-identity.png and 10-identity-top5.png.
3. **Training and confidence behaviour:** results/plots/training_curves.png, confusion_matrix.png, and calibration_coverage.png (reliability before and after temperature scaling, coverage against accuracy).
4. **Robustness under varying conditions** (brightness, low light, blur, noise, low resolution, occlusion): docs/screenshots/06-lab-robustness.png and the chart below.
5. **3D draft output** with its validation report: docs/screenshots/05-studio-mesh.png.

Blur, occlusion and low resolution cost the most accuracy

Deployed model, Top-1 (%) on 1,000 test images at three severities; bars from light to dark = mild, medium, severe



results/robustness.json (eval_P5_T25.md section 5) · 1,000 random test images, 6 conditions × 3 severities

Brightness, low light and noise cost at most 13 points even at their worst. Heavy blur and 64 px resolution roughly halve accuracy, and at 32 px Top-1 falls to 9%.

3.3 Ablation Study

We changed one component at a time against the same baseline.

Component changed	Split	Result	Change
Full pipeline: top-score box instead of largest confident box	Test, 500 images	80.60%	−11.80 pts
Backbone: ResNet-50 instead of ConvNeXt-Tiny	Test	81.58%	−10.62 pts
Quantisation: static INT8 on all layers instead of ONNX FP32	Test	86.11%	−6.09 pts
No crop: full image instead of annotated box	Test	86.15%	−5.87 pts
Backbone: EfficientNetV2-S instead of ConvNeXt-Tiny	Test	87.13%	−5.07 pts
Classical: HOG features only instead of all classical	Validation	14.40%	−2.37 pts
Annotated box instead of detector crop	Test, 500 images	91.80%	−0.60 pts

Component changed	Split	Result	Change
Quantisation: static INT8 on MatMul layers only (deployed)	Test	92.18%	−0.02 pts
Hybrid: deep + geometry instead of deep features alone	Validation	92.23%	+0.00 pts
Classical: adaptive CLAHE on	Validation	—	−0.33 pts

Three components matter most: choosing the right detected car, the ConvNeXt backbone, and quantising only the MatMul layers. The full ablation, including each classical family and each fusion, is in eval_P5_T25.md.

4. Failure Modes and Limitations

The deployed model misclassified 642 of 8,041 test images, and 196 of those errors had at least 90% confidence. Full cases with images are in docs/CLASSIFIER_FAILURES.md and docs/FAILURE_ANALYSIS.md.

Observed failure cases

Failure	What happened	Root cause	Mitigation
Wrong car chosen in a busy photo	A tiny background car outranked the main Ford Mustang Convertible 2007: box IoU 0.00 at score 1.00. This happened on 68 of 500 sampled images.	Faster R-CNN scores small distant cars as highly as the subject, so taking the top-scoring box picks the wrong car.	**Done:** the app crops the largest box scoring ≥ 0.5, chosen on validation. IoU failures fell to 4 of 500, and full-pipeline Top-1 rose from 80.6% to 92.4%.
The fix itself fails on a weak large box	On an Aston Martin V8 Vantage, a large 0.56-score box over dark studio background won: IoU 0.28.	Preferring large boxes lets a barely-passing background box beat the correct, smaller one.	Test score × area (equal on validation, 0.916 IoU) or a higher score limit on validation.

Failure	What happened	Root cause	Mitigation
Near-identical classes	A Silverado 1500 was called a 2500HD. A Chevrolet Express Cargo Van was called a GMC Savana, its badge-engineered twin.	The classes share body and cab, and differ in payload class or badge. 63.5% of all errors stay within the same make.	The ambiguous state shows the Top-5 and asks the user to confirm. Higher input resolution or part-based attention could separate trim details.
Viewpoint and blur hide the cues	A rear three-quarter view of an Aston Martin V8 Vantage became a Virage. A motion-blurred Audi TT became a TT RS.	The view or the blur removes the grille and trim that separate the models. Heavy blur alone drops Top-1 to 52.7%.	Blur and viewpoint augmentation in training; ask for a front three-quarter photo when the result is ambiguous.
Probable label noise	A Jeep labelled Compass looks like a Grand Cherokee, which is what the model answered, with 100% confidence.	The dataset's third-party test labels may contain errors.	Counted as an error. It is reported, not removed.

Limitations

- **Closed set.** The 196 classes end at 2012 model years. The confidence rule rejected 91.5% of non-car vehicles and 98.3% of other objects from Caltech-101 (AUROC 0.977), but it was not tested on newer car models, the likeliest real input.
- **Single-view 3D.** TripoSR infers the unseen sides from one photo, and the mesh has no real-world scale. The app reports both and never labels a mesh print-ready.
- **Hosting.** The free Render service has no GPU and no persistent disk, so it cannot build meshes, and accounts reset when it restarts. Full 3D runs only on the team's GPU laptop.
- **Two Top-1 figures.** The app's own preprocessing scores 92.02% against the notebook's 92.18%, 13 images apart, because of how the image is resized (results/preprocessing_check.json).

5. Deployment and Quickstart Guide

Apex3D is a FastAPI backend serving a built React frontend: one command starts the whole app on http://localhost:8000. The trained classifier (checkpoints/model_quantized.onnx, 34.6 MB) is in the repository, so recognition works straight after cloning.

Quickstart commands

```
# 1. Clone the repository and install dependencies (Python 3.12)
git clone https://github.com/sinchanaa23/Apex3D.git
cd Apex3D
pip install -r requirements.txt

# 2. Build the frontend
cd frontend && npm ci && npm run build && cd ..

# 3. Start the app (recognition, classical CV, references, metrics)
python start_server.py
# open http://localhost:8000 and sign up on first use

# 4. Optional: 3D reconstruction on an NVIDIA GPU
pip install -r requirements-ml.txt
powershell -ExecutionPolicy Bypass -File scripts/setup_triposr.ps1

# 5. Reproduce the evaluation numbers (needs the dataset)
python scripts/prepare_dataset.py
python scripts/evaluate_deployed_model.py
python scripts/build_eval_report.py
```

Deployment format

- **Runtime:** FastAPI + Uvicorn backend, React/TypeScript frontend built with Vite, and SQLite for job records.
- **Live deployment, Render free plan:**
  - a Docker web service defined by render.yaml and the root Dockerfile;
  - a lean server without PyTorch: the detector and classifier both run in ONNX Runtime, and images are preprocessed with NumPy (requirements-serve.txt);
  - server memory falls from 536 MB to 207 MB at start-up and from 816 MB to 257 MB at peak, within the free plan's 512 MB;
  - one recognition took 0.64 to 1.1 s on one CPU thread on a laptop.

- **What runs online:** login, upload and identification with the Top-5, the classical stages, references, the showcase 3D car with GLB, OBJ and STL downloads, and the Research Lab.
- **What does not:** building a new mesh needs TripoSR, so it is switched off online and runs on the GPU laptop (deploy/Dockerfile.full is for a paid plan with 2 GB or more).
- **Free-plan limits:** the service sleeps after about 15 idle minutes and takes about a minute to wake. Accounts are lost on restart because the free plan has no disk.
- **Status:** the Render deployment had not yet been run when this report was written, and the Docker image had not been built.

Student Submission Checklist

Three items still need the team before submission: the final commit hash, the live link and the AI-assistance declaration.

- [x] ~~Working GitHub repository link included, with an updated README.md (screenshots, demo video and run instructions).~~
- [x] ~~No raw code pasted into this report; only commands and file references.~~
- [x] ~~All table metrics match result files reproducible via the repository (results/ and eval_P5_T25.md).~~
- [x] ~~All team member details recorded, with USNs.~~
- [ ] Final commit hash updated in the header table, after the last push.
- [ ] Live demo link added: the onrender.com address, once the Render service is deployed and tested.
- [ ] AI-assistance declaration added. The hackathon caps AI-written content at 30% and requires AI help to be declared. This draft was written with Claude, so the team should rewrite the explanations in its own words and state the AI share honestly.